

DLMM 0.13.0

Smart Contract Security Assessment

May 2026

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	6
4	Key Findings and Recommendations	7
4.1	Incorrect System Program validation in Anchor 1.0.0	7
4.2	Potential self referral compatibility break in swap host fee account	8
5	Disclaimer	10



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers, operating systems, IoT devices, and hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple, Google, and Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of *DLMM* smart contracts, starting on May 8th, 2026, and concluding on May 28th, 2026.

Project Overview

Meteora DLMM Release 0.13.0 upgrades the on-chain program to Anchor 1.0 and Solana 3. It updates account handling, CPI calls, and account resizing for the new framework, and adds a new protocol fee claim path through the protocol fee program while keeping the existing pool and position layouts unchanged.

Audit Scope

The assessment scope contains mainly the smart contracts of the *lb-clmm* program for the *DLMM* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- DLMM
 - Codebase: <https://github.com/MeteoraAg/DLMM>
 - Pull Request: [PR-504](#)
 - Commit Hash: d402f59cda44a515d17829427da1142a26b9cfe2

The issues described in this report have been resolved by the following commit:

- DLMM
 - Codebase: <https://github.com/MeteoraAg/DLMM>
 - Commit Hash: d1ed0d046f3b7cb9111a10bd2d66f479fb024b1d

We listed the files we have audited below:

- DLMM Release 0.13.0
 - programs/lb_clmm/src/const_pda.rs
 - programs/lb_clmm/src/constants.rs
 - programs/lb_clmm/src/errors.rs
 - programs/lb_clmm/src/events.rs
 - programs/lb_clmm/src/instructions/admin/auth.rs
 - programs/lb_clmm/src/instructions/admin/mod.rs
 - programs/lb_clmm/src/instructions/claim_fee.rs
 - programs/lb_clmm/src/instructions/claim_reward.rs
 - programs/lb_clmm/src/instructions/deposit/add_liquidity_by_strategy_one_side.rs
 - programs/lb_clmm/src/instructions/deposit/add_liquidity_by_strategy.rs
 - programs/lb_clmm/src/instructions/deposit/add_liquidity_by_weight_one_side.rs



- programs/lb_clmm/src/instructions/deposit/add_liquidity_by_weight.rs
- programs/lb_clmm/src/instructions/deposit/add_liquidity_single_side_precise.rs
- programs/lb_clmm/src/instructions/deposit/add_liquidity.rs
- programs/lb_clmm/src/instructions/initialize_pool/initialize_permission_lb_pair.rs
- programs/lb_clmm/src/instructions/limit_order/cancel_limit_order.rs
- programs/lb_clmm/src/instructions/limit_order/place_limit_order.rs
- programs/lb_clmm/src/instructions/operator/claim_protocol_fee2.rs
- programs/lb_clmm/src/instructions/operator/fund_reward.rs
- programs/lb_clmm/src/instructions/operator/initialize_reward.rs
- programs/lb_clmm/src/instructions/operator/mod.rs
- programs/lb_clmm/src/instructions/operator/withdraw_ineligible_reward.rs
- programs/lb_clmm/src/instructions/operator/withdraw_protocol_fee.rs
- programs/lb_clmm/src/instructions/operator/zap_protocol_fee/process_zap_protocol_fee.rs
- programs/lb_clmm/src/instructions/permissionless_operations/initialize_bin_array.rs
- programs/lb_clmm/src/instructions/position/decrease_position_length.rs
- programs/lb_clmm/src/instructions/position/mod.rs
- programs/lb_clmm/src/instructions/rebalance/process_add_liquidity.rs
- programs/lb_clmm/src/instructions/rebalance/process_claim_fee_and_rewards.rs
- programs/lb_clmm/src/instructions/rebalance/process_resize_position.rs
- programs/lb_clmm/src/instructions/rebalance/rebalance_wrapper.rs
- programs/lb_clmm/src/instructions/swap.rs
- programs/lb_clmm/src/instructions/v2/add_liquidity_by_strategy2.rs
- programs/lb_clmm/src/instructions/v2/add_liquidity_by_weight2.rs
- programs/lb_clmm/src/instructions/v2/add_liquidity_single_side_precise2.rs
- programs/lb_clmm/src/instructions/v2/add_liquidity2.rs
- programs/lb_clmm/src/instructions/v2/claim_fee2.rs
- programs/lb_clmm/src/instructions/v2/claim_reward2.rs
- programs/lb_clmm/src/instructions/v2/initialize_customizable_permissionless_lb_pair2.rs
- programs/lb_clmm/src/instructions/v2/initialize_permissionless_lb_pair2.rs
- programs/lb_clmm/src/instructions/v2/remove_liquidity_by_range2.rs
- programs/lb_clmm/src/instructions/v2/remove_liquidity2.rs
- programs/lb_clmm/src/instructions/v2/swap2.rs
- programs/lb_clmm/src/instructions/v2/update_fees_and_reward2.rs
- programs/lb_clmm/src/instructions/withdraw/remove_all_liquidity.rs
- programs/lb_clmm/src/instructions/withdraw/remove_liquidity_by_range.rs
- programs/lb_clmm/src/instructions/withdraw/remove_liquidity.rs
- programs/lb_clmm/src/lb_access_control.rs
- programs/lb_clmm/src/lib.rs
- programs/lb_clmm/src/state/bin_array_bitmap_extension.rs
- programs/lb_clmm/src/state/dynamic_position.rs
- programs/lb_clmm/src/state/lb_pair.rs



- programs/lb_clmm/src/state/oracle.rs
- programs/lb_clmm/src/state/parameters.rs
- programs/lb_clmm/src/utls/alpha_vault.rs
- programs/lb_clmm/src/utls/mod.rs
- programs/lb_clmm/src/utls/remaining_accounts_util.rs
- programs/lb_clmm/src/utls/seeds.rs
- programs/lb_clmm/src/utls/token.rs
- programs/lb_clmm/src/utls/token2022.rs

Findings

The security audit revealed:

- 0 critical issue
- 1 high issues
- 0 medium issue
- 1 low issues
- 0 informational issue

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Incorrect System Program validation in Anchor 1.0.0	High	Fixed
02	Potential self referral compatibility break in swap host fee account	Low	Fixed



4 Key Findings and Recommendations

4.1 Incorrect System Program validation in Anchor 1.0.0

Severity: High

Status: Fixed

Target: Smart Contract

Category: Framework

Description

The program is upgraded to Anchor 1.0.0. This Anchor version has a known validation issue for typed program accounts such as `Program<'info, System>`.

In Anchor `1.0.0~1.0.1`, `Program<T>` uses the program id returned by `T::id()` to decide whether the account should be treated as a typed program or as a generic executable program. Since the System Program id is the default public key, `Program<'info, System>` may be misclassified as a generic program account. As a result, Anchor may only check that the provided account is `executable`, without enforcing that its address is the real System Program address.

```
152 // Special handling for unit type () - only check executable, not
    // program ID
153 let is_unit_type = T::id() == Pubkey::default();
154
155 if !is_unit_type && info.key != &T::id() {
156     return
        Err(Error::from(ErrorCode::InvalidProgramId).with_pubkeys(
            (*info.key, T::id())));
157 }
158 if !info.executable {
159     return Err(ErrorCode::InvalidProgramExecutable.into());
160 }
```

[lang/src/accounts/program.rs#L152-L160](#)

The affected code pattern is:

```
pub system_program: Program<'info, System>,
```

The program contains multiple instructions using this pattern for account initialization, account resizing, rent funding, and other system program operations.

Impact

An attacker may be able to pass a non System executable program where the real System Program is expected. This can affect instructions that depend on System Program validation for account creation, reallocations, rent transfers, or PDA based setup flows. The



practical impact depends on each call path, but the affected validation is security critical and applies broadly across the program.

Recommendation

It's recommended to upgrade Anchor to version 1.0.2, where this issue is fixed in this commit [commit/3eb1fb0427d4fed0e944b627aef76db651a93aab](https://github.com/project-serapion/anchor/commit/3eb1fb0427d4fed0e944b627aef76db651a93aab).

If the program continues to use Anchor 1.0.0 or 1.0.1, explicitly constrain every System Program account to the canonical System Program address, for example:

```
#[account(address = anchor_lang::system_program::ID)]
pub system_program: Program<'info, System>,
```

Mitigation Review Log

Fixed in the commit 136ec9d6c21952ff9a2034fa866e4b625333270b.

4.2 Potential self referral compatibility break in swap host fee account

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Framework

Description

Anchor 1.0 introduces `duplicate mutable account checks` for mutable account types that serialize or write back on exit, including mutable `InterfaceAccount` token accounts.

In the swap instructions, the `host_fee_in` token account and the user input token account are both `mutable` token accounts. In Anchor 0.x, the `host_fee_in` account could be the same as the user input token account, allowing the host fee to be sent back to the input token account.

However, After the Anchor 1.0 upgrade, this duplicate mutable account combination will be rejected.

Impact

If any users or integrators rely on using the input token account as the host fee account, those swaps will fail after the upgrade.

Recommendation

If self referral is supported in swap instructions, it's recommended to mark the `host_fee_in` account with `dup`.



Mitigation Review Log

Fixed in the commit [d1ed0d046f3b7cb9111a10bd2d66f479fb024b1d](#).



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

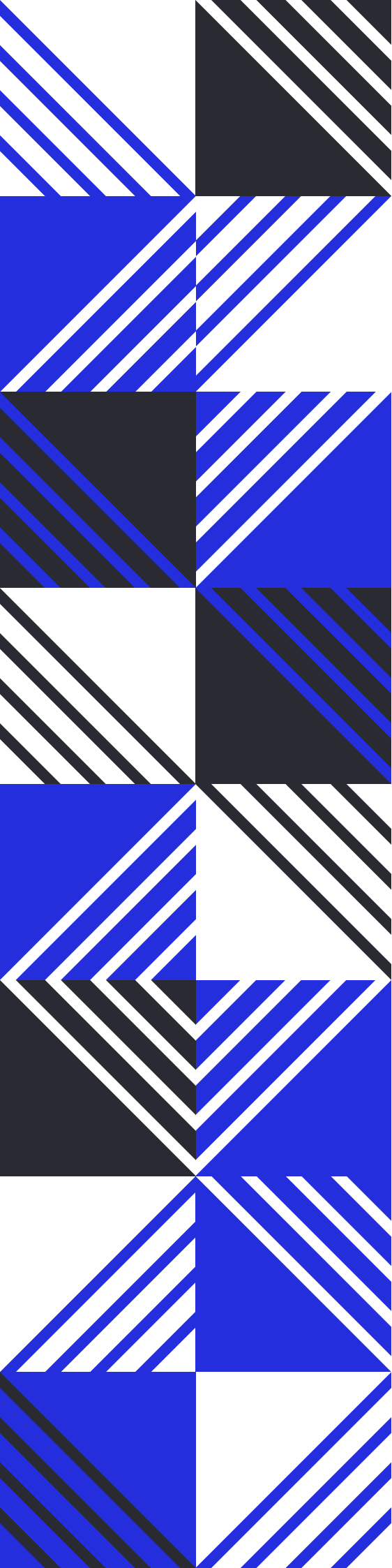
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs